

为规范GIT的使用，我司针对本项目制定GIT管理方案。

1. 代码库（repository）

本项目代码库（项目中所需的其他代码库）由阿里云的[云效Codeup](#)进行管理，其使用方法详见[云效帮助中心](#)。

本项目中使用其提供的如下功能：

- 1. 权限管理；
- 2. 代码评审；

2. 分支管理（branch）

本项目的分支采用下表的方式进行管理：

分支类型	路径	存在期	推送权限	说明
主分支	master	永久	管理员	默认分支，用于发行，包含安装部署等代码，包括软件保护和安装包制作等，是发行分支的基础
开发分支	develop	永久	管理员	用于开发调试，包含全部功能，是master分支的基础
发行分支	realese/*	永久	管理员	基于master，软件（准备） 销售 给客户的每一个版本均需要一个发行分支
客户分支	client/*	永久	管理员	类似于release分支，用于客户专用定制，包含客户专用代码，基于master分支增删功能而得
特性分支	feature/*	临时	开发者	在新功能开发时使用
热修复分支	hotfix/*	临时	开发者	客户提出软件bug，进行修正时采用的分支
开发者临时分支	developer_name/*	临时	开发者	保存开发者的工作

说明：

- 1. 推送权限为“管理员”的为受保护的分支，此类分支应使用“代码评审”的方式发起合并请求；
- 2. 临时分支，在确认已经不需要后及时清理；
- 3. release分支采用 [主版本号].[次版本号] 的形式命名，如: 1.2 ；

4. 开发者临时分支可如下书写 zhangsan/develop 或 zhangsan/client/cscec 。

3. 标签管理（tag）

代码库内的标签分为“版本标签”和“其他标签”：

1. “其他标签”为临时标签，创建者应及时清理；
2. “版本标签”为永久标签，软件发给客户的每一个版本均需指定版本标签，不可修改；

“版本标签”要求如下：

1. 应以 realese/* 或 client/* 分支为基础；
2. 版本标签应与软件代码内的版本号保持一致；
3. 采用 [V|客户名][主版本号].[次版本号].[修订日期] 的形式命名，如: V1.2.20210510 或 CSCEC1.2.20210510 ；

4. 提交（commit）

项目代码的修改要及时提交，每次提交应满足如下要求：

1. 原子化的提交，一次提交只做一件事；
2. 提交说明应与修改内容保持一致，不可挂羊头卖狗肉；
3. 保证软件、模块可编译执行，通过冒烟测试；

同时应满足下述要求。

4.1. 提交说明的书写规范（commit Message）

提交说明的书写应采用如下格式：

```
<type>(<scope>): <subject>
<BLANK LINE>
<body>
<BLANK LINE>
<footer>
```

4.1.1. Header

1. type 代表某次提交的类型，比如是修复一个 bug 还是增加一个新的 feature。支持的type（类型）如下：
 - feat：新增 feature；
 - fix：修复 bug；
 - docs：仅仅修改了文档，比如 README, CHANGELOG, CONTRIBUTE 等等；

- `style` : 仅仅修改了空格、格式缩进、逗号等等, 不改变代码逻辑;
 - `refactor` : 代码重构, 没有加新功能或者修复 bug;
 - `test` : 测试用例, 包括单元测试、集成测试等;
 - `chore` : 改变构建流程、或者增加依赖库、工具等;
 - `revert` : 回滚到上一个版本;
 - `save` : 为了保存更改而发起的临时提交, 在后续需对此次提交进行合并处理, 不应该出现在提交历史中;
2. `scope` 用于说明 `commit` 影响的范围, 比如数据层、控制层、视图层等等, 视项目不同而不同;
 3. `subject` 是 `commit` 目的的简短描述, 不超过 50 个字符;
 - 以动词开头, 使用第一人称现在时, 比如 `change`, 而不是 `changed` 或 `changes`;
 - 第一个字母小写;
 - 结尾不加句号 (.)。

4.1.2. Body

`Body` 部分是对本次 `commit` 的详细描述, 可以分成多行。

1. 使用第一人称现在时, 比如使用 `change` 而不是 `changed` 或 `changes`;
2. 应该说明代码变动的动机, 以及与以前行为的对比。

范例 1:

```
More detailed explanatory text, if necessary. Wrap it to
about 72 characters or so.
Further paragraphs come after blank lines.
- Bullet points are okay, too
- Use a hanging indent
```

范例 2:

主体内容: 更详细的说明文本。 需要描述的信息包括:

- * 为什么这个变更是必须的? 它可能是用来修复一个 `bug`, 增加一个 `feature`, 提升性能、可靠性、稳定性等等
- * 他如何解决这个问题? 具体描述解决问题的步骤
- * 是否存在副作用、风险?

尾部: 如果需要的化可以添加一个链接到 `issue` 地址或者其它文档, 或者关闭某个 `issue`。

4.1.3. Footer

`Footer` 部分只用于两种情况:

1. 不兼容变动如果当前代码与上一个版本不兼容, 则 `Footer` 部分以 `BREAKING CHANGE` 开头, 后面是对变动的描述、以及变动理由和迁移方法。

BREAKING CHANGE: isolate scope bindings definition has changed.

To migrate the code follow the example below:

Before:

```
scope: {  
  myAttr: 'attribute',  
}
```

After:

```
scope: {  
  myAttr: '@',  
}
```

The removed ``inject`` wasn't generally useful for directives so there should be no code using it.

2. 关闭issue

如果当前 commit 针对某个 issue，那么可以在 Footer 部分关闭这个 issue。

Closes #234

也可以一次关闭多个 issue。

Closes #123, #245,

4.2. revert

如果当前 commit 用于撤销以前的 commit，则必须以 `revert:` 开头，后面跟着被撤销 Commit 的 Header。

```
revert: feat(pencil): add 'graphiteWidth' option  
This reverts commit 667ecc1654a317a13331b17617d973392f415f02.
```

Body 部分的格式是固定的，必须写成 `This reverts commit <hash>.`，其中的 `hash` 是被撤销 commit 的 SHA 标识符。

4.3. amend

基于 **save** 提交的 `rebase:` 前一次提交为 **save** 提交，直接使用 **amend** 合并上次提交

5. 推送 (push)

所有本地的代码修改，每天均应及时推送到远端代码库，以备份代码。

对于受保护的分支（如 `master`、`release` 等），不可直接推送，应发起合并请求（代码评审）。

6. 合并（merge/rebase）

rebase 的合并方式可以使用，但不可应用于已存在于 master、develop、release 的，或已被他人使用的提交。

本项目使用阿里云的代码评审功能，评审操作方式详见[云效帮助中心](#)，对于受保护的分支，应使用“代码评审”的方式发起合并请求，并：

1. 仅支持 fast-forward-only 类型的合并；
2. 评审通过时，开发者应尽可能地（多个提交只做一件事）压缩提交（squash）后，再合并；
3. 评审由管理员执行；
4. 评审的发起可使用[AGit-Flow](#)工具来提高效率。